

document are sourced from repository files only.
No speculation. No future state projection.
Assessment Date: 2026-00-02
Where evidence does not exist, it is stated explicitly.

CerbaSeal

CTO-Level Pilot Readiness Assessment

Prepared for: Olivia Aréchiga, Co-Founder & CTO, Line Axia Consulting

All claims in this document are sourced from repository files only.
Repository: CerbaSeal-Core v0.1.0 — Lamont Labs
No speculation. No future state projection.
Assessment Date: 2026-00-02
Where evidence does not exist, it is stated explicitly.

PART 1 — OLIVIA'S QUESTIONS

QUESTION: Q1 — Where is CerbaSeal relative to external deployment? What is the gap to pilot-ready?

CURRENT STATE:

"VeraSeal" does not appear anywhere in this repository. There is no component, module, file, or reference using that name. The enforcement core is CerbaSeal itself — specifically ExecutionGateService (src/services/execution/execution-gate-service.ts). There is no sub-component called VeraSeal.

CerbaSeal-Core v0.1.0 is a deterministic enforcement library and browser demo. Its own documentation states it is "a minimal reviewable enforcement proof package" and "not a finished standalone deployable product." It runs on Node.js/TypeScript with a single runtime dependency (tsx).

Implemented and working: 12-invariant execution gate, ALLOW/HOLD/REJECT outcomes, fail-closed behavior, SHA-256 hash-chained audit log, evidence bundle generation, replay verification, diagnostic reports, 323 passing tests across 15 test files, browser portal (/review, /pilot, /security, /deployment), 15/15 audit checks in CI.

Not implemented: persistent storage (in-memory, lost on restart), cryptographic signing, identity attestation, policy content evaluation, client-specific workflow binding, production infrastructure hardening, no third-party security review.

EVIDENCE:

docs/current_maturity.md · docs/pilot/pilot-readiness-brief.md · docs/status/current-state.md · docs/deployment/eu-pilot-deployment-posture.md · src/services/execution/execution-gate-service.ts · package.json (v0.1.0, tsx only). No file in the repository contains the string "VeraSeal."

GAPS:

VeraSeal as a named component: No evidence found — does not exist. Production deployment: No evidence found. Signed pilot client: No evidence found. Third-party security review: No evidence found.

PILOT-READY ANSWER:

"CerbaSeal is the enforcement core — there is no separate component called VeraSeal in the current build. The core enforcement logic is implemented, adversarially tested, and documented. What we have today is a controlled technical proof, not a production deployment. Pilot-ready means: one workflow, one client, controlled environment, no production execution. The gap is the working agreement, the deployment environment, and persistent storage."

CONFIDENCE: High — based on direct repository evidence

QUESTION: Q2 — Can CerbaSeal be deployed in a client-controlled or EU-hosted environment?

CURRENT STATE:

The architecture supports EU/client-controlled deployment by design. CerbaSeal has no outbound network calls, no external API dependencies, no database, and no framework dependencies. Four deployment modes documented: (1) Embedded library, (2) Internal HTTP service, (3) Sidecar, (4) Air-gapped evaluation. The EU posture document states explicitly:

"CerbaSeal does not require outbound network access. CerbaSeal does not call external APIs. CerbaSeal does not transmit data to third-party services." Mode C (client-controlled environment) is the documented preferred pilot approach for EU data residency.

What does NOT exist: a deployed EU instance, a tested deployment playbook, infrastructure-as-code, a DPA template, legal review of EU requirements.

EVIDENCE:

docs/deployment/deployment-modes.md · docs/deployment/eu-pilot-deployment-posture.md · package.json (tsx only) · src/services/execution/execution-gate-service.ts (no network calls)

GAPS:

Tested EU deployment: No evidence found. Containerization / Dockerfile: No evidence found. Infrastructure-as-code: No evidence found. DPA template: No evidence found. EU legal review: No evidence found.

PILOT-READY ANSWER:

"Yes, the architecture supports this — no external calls, no cloud-vendor dependencies, no data transmitted outside the

deployment environment. The deployment model that achieves data residency is client-controlled hosting: deploy the Node.js package inside your own EU infrastructure. We have not yet executed a EU deployment; the path exists but has not been walked. A DPA and environment review are prerequisites we have documented."

CONFIDENCE: High for architectural claim / Medium for operational readiness

QUESTION: Q3 — What would your operational role look like during a pilot?

CURRENT STATE:

The pilot operations model is fully documented (docs/pilot-operations-model.md). Week 1: scoping, baseline deployment, success criteria, workflow mapping, authority matrix. Ongoing: email support, weekly 30-minute review calls, tracked issue queue, change log, decision log.

Response times: P1 (system unusable) same business day · P2 (major pilot impact) 24 hours · P3 (general issue) 3 business days · P4 (enhancement) weekly review.

Founder unavailability up to 48 hours: demo environment, enforcement docs, security docs, review portal, proof snapshots, invariant registry — all accessible without founder. Pauses during unavailability: new issue investigation requiring code changes, configuration changes, scheduled calls.

EVIDENCE:

docs/pilot-operations-model.md (full SLA model) · docs/operations/solo-support-risk-reduction.md · docs/operations/pilot-safe-mode.md

GAPS:

Second engineer / backup escalation path: No evidence found. Managed hosted service: No evidence found. 24/7 support: explicitly excluded.

PILOT-READY ANSWER:

"My role is highest at week one — scoping, deployment, baseline scenarios, success criteria. After that: weekly check-in calls, tracked issue queue, email support. CerbaSeal is self-explaining — every outcome generates a reason code and diagnostic trace. I am a solo founder and I have documented that honestly. I will not overcommit. If Line Axia needs a larger support model, that becomes part of the engagement structure."

CONFIDENCE: High — fully documented in repository

QUESTION: Q4 — What is your realistic capacity for a pilot engagement?

CURRENT STATE:

No evidence in this repository addresses Jesse Lamont's personal schedule, current portfolio engagements, or capacity constraints. This is a personal operations question, not a technical question.

EVIDENCE:

No evidence found in repository.

GAPS:

No repository evidence. Requires Jesse's direct answer.

PILOT-READY ANSWER:

"I do not have other active pilot deployments right now. Week one is the highest-touch period. After that the weekly model is sustainable alongside ongoing development. I will give you an honest read on timing on the call."

CONFIDENCE: Low — no repository evidence; Jesse answers directly

QUESTION: Q5 — Are there open-source components or third-party dependencies we should be aware of?

CURRENT STATE:

Runtime dependencies (package.json): tsx ^4.21.0 (MIT license) — TypeScript execution engine. This is the ONLY runtime dependency. Dev dependencies (build/test time only, not in deployment): typescript ^5.6.3 (Apache 2.0), vitest ^2.1.8 (MIT), @types/node ^22.10.2 (MIT). No framework, no database drivers, no cloud SDKs, no auth libraries, no external service connections.

EVIDENCE:

package.json (complete dependency list) · src/ (no external imports beyond Node.js built-ins)

GAPS:

No SBOM produced. No formal license audit. tsx has not been independently security-audited within this repository.

PILOT-READY ANSWER:

"The runtime deployment has one external dependency: tsx (MIT-licensed TypeScript runtime). No framework dependencies, no database drivers, no cloud SDK calls, nothing that reaches outside the deployment environment. A formal SBOM would be straightforward given the minimal surface."

CONFIDENCE: High — package.json is definitive

QUESTION: Q6 — How do you want to be compensated for your technical contribution?**CURRENT STATE:**

No repository document specifies compensation, pricing, revenue sharing, or fee structure. The pilot readiness brief explicitly states: "This brief does not include pricing, revenue terms, or commercial commitments. Those require a separate working agreement."

EVIDENCE:

No evidence found in repository.

GAPS:

Pricing/compensation model: No evidence found. Not within repository scope by design.

PILOT-READY ANSWER:

"The repository does not specify pricing — that is deliberate. My position: I want a structure that is fair to both sides, reflects actual value, and does not create misaligned incentives. Fixed fee, percentage of engagement, or hybrid — I am open to discussion. What matters most to me is that scope boundary and change request definition are agreed before we start."

CONFIDENCE: Low — no repository evidence; Jesse answers directly

QUESTION: Q7 — Does the Line Axia / Lamont Labs division of responsibility feel workable?**CURRENT STATE:**

The repository reflects this division structurally. The technical core is self-contained with no client-relationship artifacts, no commercial materials beyond the readiness binder, and no regulatory framing built in. Documentation is written for technical reviewers — consistent with a commercial partner handling client-facing positioning.

EVIDENCE:

docs/pilot/pilot-readiness-brief.md (scope boundaries) · docs/pilot-operations-model.md (technical support scope only) · docs/02-scope-boundary.md

GAPS:

Formal partnership agreement: No evidence found.

PILOT-READY ANSWER:

"Yes, that division maps to how the repository is structured. The technical layer is my scope — enforcement, evidence, deployment, bug fixes. The line I would want formalized: 'configuration within scope' vs. 'custom development outside scope' — that distinction will come up during a pilot and must be agreed in writing before we start."

CONFIDENCE: High for technical alignment / Low for formal agreement (none exists)

QUESTION: Q8 — Are there any other IP considerations visible from the repository?**CURRENT STATE:**

The repository has no LICENSE file. There is no open-source license declared, no copyright notice in source files, no CLA. The repo is named CerbaSeal-Core under the Lamont-Labs GitHub organization. No third-party code has been copied in; all source appears original. The only third-party code is through npm dependencies (all MIT or Apache 2.0). The absence of a LICENSE file means the repository is technically "all rights reserved" by default — consistent with proprietary ownership but not explicitly declared. Evidence artifact ownership (who owns audit chains and bundles produced during a client pilot) is not resolved in any document.

EVIDENCE:

Repository root (no LICENSE file) · package.json (no license field) · docs/pilot/pilot-readiness-brief.md ("evidence ownership" listed as required pre-pilot item)

GAPS:

LICENSE file: No evidence found. Copyright notices in source: No evidence found. Evidence artifact ownership during pilot: identified as gap, not resolved.

PILOT-READY ANSWER:

"Lamont Labs owns CerbaSeal. No open-source license — consistent with proprietary ownership but should be formalized. The key working-agreement item: who owns evidence bundles and audit chains produced during the pilot. That is already on our prerequisites list."

CONFIDENCE: High for observation / Low for formal IP clarity

QUESTION: Q9 — Would a 12-month EU pilot exclusivity arrangement be operationally reasonable?

CURRENT STATE:

No repository document addresses exclusivity or right of first refusal. From a technical standpoint: the system is a library with no client-specific code paths baked in. Nothing in the architecture prevents serving multiple clients from the same codebase.

EVIDENCE:

No evidence found in repository.

GAPS:

Exclusivity terms: No evidence found. Commercial decision, not a technical one.

PILOT-READY ANSWER:

"There is nothing in the architecture that creates a technical dependency on exclusivity. Whether 12 months of EU exclusivity makes sense is a business judgment. What I would want defined: what counts as 'EU' for exclusivity purposes, what conversion triggers the right-of-first-refusal, and what ends the exclusivity if the engagement is not active."

CONFIDENCE: Low — no repository evidence; Jesse answers directly

QUESTION: Q10 — What else should exist in a working agreement before a pilot begins?

CURRENT STATE:

docs/pilot/pilot-readiness-brief.md lists: commercial terms, ownership of evidence records, liability boundary, support period and scope, payment and billing, data processing agreement (if applicable), version freeze and update process.

docs/deployment/eu-pilot-deployment-posture.md adds: client deployment environment security review, data residency review, third-party security review, legal review of evidence retention, definition of support boundary and escalation path, persistent audit storage integration.

docs/pilot-operations-model.md adds: signed scope document, authority matrix, pilot success criteria, issue tracking setup.

Additional gaps not addressed in any document: governing law and jurisdiction, change-of-control clause, change request pricing structure.

EVIDENCE:

docs/pilot/pilot-readiness-brief.md · docs/deployment/eu-pilot-deployment-posture.md · docs/pilot-operations-model.md

GAPS:

Governing law clause: No evidence found. Change-of-control clause: No evidence found. Change request fee structure: No evidence found.

PILOT-READY ANSWER:

"At minimum: signed scope document, evidence ownership definition, liability boundary, support terms, DPA if EU data is involved, version freeze policy. Beyond what is documented: what constitutes a change request and how those are priced, and governing law. The cleaner we are on paper before we start, the better the pilot goes."

CONFIDENCE: High for repository-evidenced items / Low for unaddressed items

PART 2 — CTO REVIEW PACK

SECTION 1 — TECHNICAL READINESS ASSESSMENT

Enforcement Core — Strengths

- 12 invariants fully implemented, documented, and mapped to tests (architecture/invariants/invariant-registry.yaml)
- Gate sequence deterministic: 13 checks inside evaluate(), INV-06 enforced downstream via WeakSet at

EvidenceBundleService

- Seven targeted security fixes applied and tested following a hostile audit (docs/status/current-state.md)
- Fail-closed is real: unexpected exceptions produce controlled REJECT — not silent pass (test/security/fail-closed.test.ts)

- AI non-authority (INV-05) is a hard invariant, not a policy flag — "not configurable at runtime"
- GateResult forgery blocked by WeakSet registry — cannot be satisfied by constructing a GateResult outside evaluate()

Enforcement Core — Weaknesses

- loggingReady, trustState, prohibitedUse are all caller-supplied booleans — trusted without independent verification
- approvedAt has no expiry check, no timestamp format validation
- immutableSignature content not cryptographically verified — any non-empty string passes
- actorAuthorityClass range: only "ai" is specifically matched; unknown values not explicitly rejected
- Objects not frozen at runtime between evaluate() and createBundle() — mutation window exists

Blockers & Pilot Risks

- No blocker to controlled technical pilot use
- Persistent storage required before any production use
- Cryptographic signing required before legal-weight evidence claims
- Caller-declared fields create dependency on upstream caller trustworthiness — manageable in controlled pilot, requires architecture review in production

Test Maturity — 323 passing tests, 0 failing, 15 test files

Test File	Count	Coverage Area
adversarial-integrity.test.ts	66	Bypass attempts, forged inputs
execution-gate-service.test.ts	19	Core invariant enforcement
enforcement-loop.snapshot.test.ts	41	Snapshot regression
security/misuse-scenarios.test.ts	27	Real-world misuse patterns
security/contextual-boundary.test.ts	25	Enforcement limits
integration/review-portal-routes.test.ts	61	Portal route coverage
integration/browser-demo-routes.test.ts	28	Demo server routes
integration/support-readiness.test.ts	23	Support layer
integration/external-signal-examples.test.ts	16	Signal examples
security/non-forgery.test.ts	2	GateResult forgery prevention
security/fail-closed.test.ts	2	Exception handling
audit-evidence-export.test.ts	6	Evidence export
diagnostic-report-service.test.ts	5	Diagnostics
integration/full-flow.test.ts	1	End-to-end
integration/system-integration.test.ts	1	System integration

Security-focused tests: 66 adversarial + 27 misuse + 25 boundary

Weaknesses: full-flow.test.ts and system-integration.test.ts each have 1 load or stress testing.

Deployment Readiness

- Current: Replit-hosted demo at cerbaseal.replit.app — suitable for test
- 4 deployment modes documented: embedded library, internal HTTP
- Architecturally portable: no external calls, no framework dependencies
- No Dockerfile or container configuration
- No infrastructure-as-code
- No deployment playbook or runbook
- No persistent storage integration
- A first pilot deployment would be the first time this package is deployed

Documentation Quality

- docs/security/security-review-brief.md: most honest self-assessment of limitations, does not overclaim
- docs/current_maturity.md: explicitly states what is real vs. not yet built
- docs/09-trust-boundary-and-limitations.md: documents structural trust boundaries
- INCONSISTENCY: docs/current_maturity.md states 372 passing tests, 323. Must be corrected before external sharing.
- No deployment runbook. No SBOM produced.

SECTION 2 — PILOT DEPLOYMENT PLAN

	Today	First Pilot	Production (Future)
Runtime	Node.js + tsx In-memory state	Node.js + tsx Client infrastructure	Node.js + tsx Hardened environment
Hosting	Replit (demo only)	Client-controlled (EU-compatible)	Managed / client-operated
Audit log	In-memory Lost on restart	In-memory (pilot eval) Persistent: must be built	Persistent storage (not yet designed)
Deployment	No reproducible deployment outside Replit	First client deployment (untested)	Versioned, containerized
Client data	None	Yes — controlled pilot environment only	Production decisions

What can be reused from today

- Full enforcement core (ExecutionGateService)
- Evidence bundle generation and audit log
- Diagnostic report generation and operator
- All 323 tests, 15 audit checks, proof snapshots
- Portal documentation (/review, /pilot, /security)

What must be built for first pilot

- Persistent audit log integration (evidence not reusable)
- GovernedRequest construction for the spec
- Deployment environment: container, VM, cloud
- API surface if deploying as Mode B (internal)

SECTION 3 — OPERATIONAL SUPPORT PLAN

Source: docs/pilot-operations-model.md

Incident Priority & Response Times

Priority	Definition	Response
P1 — System unusable	Pilot environment down or gate not producing outcomes	Same business day
P2 — Major pilot impact	Enforcement behavior contradicts documented invariants	Within 24 hours
P3 — General issue	Documentation error, config question, clarification	Within 3 business days
P4 — Enhancement	Feature or scope expansion outside pilot definition	Weekly planning

Founder Dependency — What pauses vs. what does not

If unavailable up to 48 hours — these continue without pause:

- Demo environment, enforcement docs, security docs
- pnpm audit:repo — re-run all 15 audit checks independently
- pnpm verify:proof — verify stableChecksum independently
- Evidence bundles and diagnostic outputs readable

These pause during unavailability:

- New issue investigation requiring live code changes
- Configuration changes to pilot environment
- Scheduled review calls (rescheduled on return)

Beyond 48 hours: No contingency is documented.

SECTION 4 — EU HOSTING FEASIBILITY ASSESSMENT

Implemented

- No outbound network calls in any enforcement code path
- No external API dependencies, no cloud SDK, no hardcoded infrastructure assumptions
- Single runtime dependency (tsx, MIT licensed, EU-deployable)
- All enforcement logic self-contained in Node.js package

Feasible — not yet executed

- Mode C deployment (client-controlled EU environment) documented as preferred pilot mode
- Package deployable wherever Node.js runs — no architectural barrier
- Client selects and controls hosting environment and region

Unknown / Requires Resolution

- No EU deployment tested or executed
- No data processing agreement template exists
- Legal review of EU jurisdiction requirements: not completed
- GDPR applicability to evidence bundle contents: not assessed
- Persistent audit storage in EU-controlled infrastructure: not designed

Risk summary: Low architectural risk (code does not need to change). Medium operational risk (EU deployment never executed). High legal/compliance risk (GDPR, DPA obligations unaddressed — requires legal input before live pilot with real client data).

SECTION 5 — OPEN SOURCE & DEPENDENCY INVENTORY

Runtime Dependencies

Dependency	Version	License	Purpose	Risk	Pilot Impact
tsx	^4.21.0	MIT	TypeScript execution runtime	Low	Required in all deployments

Build / Dev Dependencies (not present in deployment artifact)

Dependency	Version	License	Purpose	Risk
typescript	^5.6.3	Apache 2.0	Compiler	Low
vitest	^2.1.8	MIT	Test framework	Low
@types/node	^22.10.2	MIT	Node.js types	Low

Notable: The enforcement core has extra dependencies for enterprise governance tooling and

SECTION 6 — COMMERCIAL DISCUSSION RECOMMENDATIONS

Not legal advice. Technical founder perspective.

- PILOT FEES: Fixed fee preferred over hourly. Suggest: fixed fee for Week 1 scoping + 0.5 week of development.
- DEPLOYMENT FEES: Distinguish initial deployment from ongoing support.
- SUPPORT OBLIGATIONS: Reproduction of evidence without paraphrasing creates disputes.
- IP OWNERSHIP: Lamont Labs retain all IP rights in the client — this builds trust.

- SCOPE BOUNDARIES: Attach pilot s updated.
- EXCLUSIVITY: If pursued — define E conversion. 12 months is a long commitm

SECTION 7 — PILOT RISK REGISTER

Risk	Sev	Like- lihood	Current Mitigation	Remaining Gap
Solo founder unavailability	High	Med	48-hour self-service model; portal, proof snapshot, audit scripts all accessible	No secondary resource; no contingency beyond 48 hours
Persistent storage absent	High	Certain	Documented gap; pilot-safe uses in-memory only	Must be built before production; no plan defined
No third-party security review	High	Certain	Security brief documents limitations honestly	External firm not engaged; required before production
Caller-supplied field trust	High	Med	Well-documented; caller assumed trusted	No runtime verification; depends on integration architecture
First-ever client deployment	High	Certain	Deployment modes documented; no external calls simplify	First real deployment adds unknowns; no runbook
Cryptographic signing absent	Med	Certain	Hash-chaining provides structural integrity	Legal-weight evidence requires signing; not yet built
Test count inconsistency	Low	Certain	Both docs readable	current_maturity.md (372) vs current-state.md (323) — correct before
No LICENSE file	Med	Certain	IP understood informally	Signing externally absent; remedy before any agreement
Scope creep	Med	High	Boundaries documented; out-of-scope explicit	Agreement must reproduce scope verbatim; none signed
Evidence chain fabrication	Med	Low	Documented; hash chain proves consistency not origin	Requires HMAC or external attestation to close
No npm publication	Low	Certain	Local install works for controlled pilot	Manual installation process; not scalable

SECTION 8 — QUESTIONS OLIVIA IS MOST LIKELY TO ASK NEXT

Ranked highest to lowest probability based on rep

1. "Can you show me the system running on infrastructure we control, not Replit?"

Gap: No client deployment has been executed.

2. "What happens to the evidence if the process restarts?"

Gap: In-memory audit log — documented gap, no

3. "Who else has reviewed the security of this system?"

Gap: No third-party security review completed.

4. "What happens if you're hit by a bus?"

Gap: 48-hour clause documented; nothing beyond

5. "What version are we freezing for the pilot, and how do you handle updates?"

Gap: Version freeze policy not implemented.

6. "Show me the GDPR analysis for this system."

Gap: No GDPR analysis exists in the repository.

7. "What does 'policy pack reference' actually mean?"

Gap: Gate requires policyPackRef but does not e

8. "Do you have cyber liability insurance?"

Gap: No evidence in repository. Jesse answers d

9. "What does the working agreement look like?"

Gap: Prerequisites documented; no draft agreem

10. "How do we know the audit chain wasn't fabricated?"

Gap: SHA-256 without HMAC — acknowledged

PART 3 — EXECUTIVE SUMMARY (ONE-PAGE FOUNDER BRIEFING)

Written for Jesse Lamont. Plain English. Evidence-based only. Maximum honesty.

WHAT CERBASEAL ACTUALLY IS TODAY

CerbaSeal is a working enforcement engine for AI-assisted workflows. When a system asks "should this AI action be allowed to execute?" — CerbaSeal answers that question and produces a verifiable record of the answer. The enforcement logic is real, adversarially tested, and honest about where it stops. One runtime dependency. No external APIs, no database, no cloud vendor lock-in. Currently hosted as a demonstration on Replit — not connected to any client system.

WHAT IS GENUINELY PILOT-READY

- Enforcement core: 12 invariants, fully implemented. Fail-closed is real. AI cannot authorize its own proposals — hard rule, not a policy flag.
- 323 tests passing including 122 security-focused tests probing bypass attempts and boundary conditions.
- Documentation: unusually honest — security review brief, maturity doc, and trust boundary doc say clearly what works and what does not.
- Pilot shape defined: one client, one workflow, one decision path. Operations model fully documented.
- Audit trail: every outcome produces verifiable evidence; proof snapshot independently verifiable by anyone.

WHAT IS NOT PILOT-READY

- Persistent storage: audit log disappears on server restart. Fine for a demo. Not fine for a client pilot.
- Cryptographic signing: evidence chain proves internal consistency, not origin. Not legal-weight evidence yet.
- Deployment: CerbaSeal has never been deployed outside Replit. The first client deployment is a first.
- Third-party security review: has not happened. All security review to date is internal.
- Signed agreement with any client: does not exist.

WHAT OLIVIA IS MOST LIKELY EVALUATING

Whether you can be trusted as a technical partner. Specifically: Is the enforcement core real? (Yes.) Is the documentation honest? (Yes.) Can this survive a pilot? (Conditionally yes.) Is this production-ready? (No — and you should say so.) She will probe the gaps. The most likely ones: in-memory storage, no third-party security review, no EU deployment executed, solo-founder operational risk.

WHAT TO CONFIDENTLY SAY

- "CerbaSeal enforces AI governance at the execution boundary. The logic is tested, honest, and designed to fail safe — if something is wrong, it blocks, not permits."
- "Pilot-ready means: one workflow, one client, controlled environment, no production execution."
- "No external calls, no cloud vendor dependency, no data leaving the client environment. EU deployment is architecturally straightforward."
- "I am a solo founder. I have documented what that means and built the system so it does not require me to interpret every outcome."
- "The system is not production-hardened today. I know exactly what that means and what it takes to get there."

WHAT NOT TO CLAIM

- Do not claim "VeraSeal" is a component. That name does not exist in the repository.
- Do not claim production-readiness for consequential decisions.
- Do not claim the audit chain is cryptographically signed or legally attested — it is hash-linked, not signed.
- Do not claim a third-party security review has been completed.
- Do not claim the system protects against malicious caller-supplied fields (loggingReady, trustState, prohibitedUse are all caller-declared).
- Do not claim any existing client or existing commercial agreement. None exists.

BIGGEST STRENGTHS

- Enforcement core is architecturally real and adversarially tested. Rare at pre-pilot stage.
- Documentation is honest. A CTO who reads the security review brief will trust you more, not less.

- Zero external runtime dependencies beyond tsx. Genuine differentiator for EU data residency.
- "AI cannot authorize its own proposals" is a hard invariant with 122 security tests behind it — not a promise in a slide.

BIGGEST RISKS

- Persistent storage does not exist. Put it on the pilot requirements list before agreeing to anything.
- Solo-founder operational risk. Be honest. Document the 48-hour model and your plan if the engagement grows.
- First client deployment is untested. Architecture supports EU hosting; the operational path has not been walked.
- No third-party security review. For a governance tool in a regulated context, acknowledge it and name a plan.

RECOMMENDED POSITION FOR THE CALL

Lead with what is real: a working, honestly documented enforcement core, ready for a controlled technical evaluation. Be specific about the pilot shape: one workflow, one client, no production execution. Name the prerequisites — working agreement, persistent storage, deployment environment — and frame them as things to resolve together, not problems to hide.

The competitive position is not "we are production-ready." The competitive position is: "We are honest about what we are, we have built the enforcement logic that most governance products only describe, and we are ready to prove it in a controlled environment with defined success criteria."

That is a stronger position than overclaiming. Olivia will test the claims. The ones above will hold.

